

Assignment 3: GitHub Developer Explorer

Course: Backend Development with Node.js

Module: JavaScript Fundamentals - Asynchronous JavaScript

Deadline: Aug 10, 6:00 midnight

Project Overview

Imagine you are building a small developer-insight tool for a technical recruitment team. The team wants to enter a GitHub username and quickly view useful public information about that developer and their repositories.

Your task is to build a GitHub Developer Explorer using Vanilla JavaScript and the public GitHub REST API. The application will request real data from GitHub, wait for asynchronous responses, validate the responses, transform repository data, and display a clear developer summary.

This project focuses mainly on asynchronous JavaScript. You are not required to build a complex user interface. A simple HTML page with an input, button, loading message, and results section is enough. The quality of your JavaScript logic is more important than the visual design.

Do not use Express.js, Axios, React, Vue, external npm packages, or a database. Use the browser Fetch API and standard JavaScript only.

Learning Objectives

By completing this assignment, you should be able to:

- Consume a real REST API using `fetch()`.
- Understand that `fetch()` returns a Promise.
- Use `async` functions and `await` to write asynchronous code.
- Run independent asynchronous requests concurrently using `Promise.all()`.
- Convert an HTTP response body from JSON into JavaScript data using `response.json()`.
- Check HTTP responses using `response.ok` and `response.status`.
- Use `try...catch` and `throw new Error()` to handle failures.
- Work with arrays of objects returned by an API.
- Use `find()`, `filter()`, `map()`, `reduce()`, `sort()`, `some()`, and other array methods where appropriate.
- Use destructuring, optional chaining, nullish coalescing, and template literals.
- Use `Date` to work with timestamps returned by an API.
- Organize code into multiple ES modules using `import` and `export`.

GitHub REST API

GitHub provides a REST API that allows applications to request public GitHub information. For this project, you will use two main endpoints. Both endpoints can be used without authentication when requesting public information.

Base API URL:

`https://api.github.com`

The value {username} in the URLs below is a placeholder. Replace it with the actual GitHub username entered by the user. For example, if the username is octocat, {username} becomes octocat.

API Endpoint 1 - Get a User Profile

Request URL:

GET `https://api.github.com/users/{username}`

Example:

`https://api.github.com/users/octocat`

Purpose:

Use this endpoint to retrieve publicly available profile information for one GitHub account. A successful request returns one JavaScript object after the JSON response is parsed.

Important profile fields for this assignment:

Field	Meaning	Example / Type
login	GitHub username	"octocat" - string
name	Display name; may be null	string or null
avatar_url	URL of profile image	string
html_url	Public GitHub profile URL	string
bio	Profile biography; may be null	string or null
company	Company; may be null	string or null
location	Location; may be null	string or null
public_repos	Number of public repositories	number
followers	Follower count	number
following	Following count	number
created_at	Account creation timestamp	ISO date string
updated_at	Last profile update timestamp	ISO date string

Important: Do not assume optional fields always contain text. A developer may not have a bio, company, or location, so these fields can be null. Handle missing values gracefully, for example by displaying "Not provided".

Example Profile Response

The real response contains additional fields. The simplified example below shows the fields most relevant to this assignment:

```
{
  "login": "octocat",
  "name": "The Octocat",
  "avatar_url": "https://...",
  "html_url": "https://github.com/octocat",
  "bio": "A GitHub example account",
  "company": "GitHub",
  "location": "San Francisco",
  "public_repos": 8,
  "followers": 10000,
```

```

    "following": 9,
    "created_at": "2011-01-25T18:44:36Z",
    "updated_at": "2026-01-01T12:00:00Z"
  }
}

```

After `response.json()`, you should treat this result as a normal JavaScript object. For example, `profile.login` gives the username and `profile.followers` gives the follower count.

API Endpoint 2 - List a User's Public Repositories

Request URL:

```
GET https://api.github.com/users/{username}/repos
```

Recommended project URL:

```
GET https://api.github.com/users/{username}/repos?per_page=100&sort=updated
```

Example:

```
https://api.github.com/users/octocat/repos?per_page=100&sort=updated
```

Purpose:

Use this endpoint to retrieve repositories owned by the selected user. After `response.json()`, the result is an array. Each item in that array is a repository object.

The `per_page` parameter controls how many repositories are returned on one page. GitHub allows up to 100 results per page. Without it, the default is 30. For this assignment, `per_page=100` is recommended so that most student test accounts can be analyzed with one request.

If a user owns more than 100 public repositories, the first response will not contain every repository. Full pagination is optional for this assignment and can be implemented as a bonus feature.

Important Repository Fields

Field	Meaning
<code>id</code>	Unique repository ID
<code>name</code>	Repository name
<code>full_name</code>	Owner/repository name
<code>html_url</code>	Public repository page
<code>description</code>	Description; can be null
<code>fork</code>	Whether the repository is a fork
<code>language</code>	Primary detected language; can be null
<code>stargazers_count</code>	Number of stars
<code>forks_count</code>	Number of forks
<code>open_issues_count</code>	Open issue count
<code>size</code>	Repository size reported by GitHub
<code>archived</code>	Whether the repository is archived
<code>created_at</code>	Creation timestamp
<code>updated_at</code>	Last update timestamp
<code>pushed_at</code>	Most recent push timestamp

Example Repository Response

The repository endpoint returns an array similar to the following simplified example:

```

[
  {
    "id": 1,
    "name": "project-one",
    "html_url": "https://github.com/example/project-one",
    "description": "Example project",
    "fork": false,

```

```

    "language": "JavaScript",
    "stargazers_count": 25,
    "forks_count": 4,
    "open_issues_count": 2,
    "archived": false,
    "created_at": "2024-01-10T10:00:00Z",
    "updated_at": "2026-07-01T10:00:00Z"
  },
  {
    "id": 2,
    "name": "python-tool",
    "language": "Python",
    "stargazers_count": 12,
    "forks_count": 1,
    "fork": false
  }
]

```

Your job is not simply to print this raw array. Your application must transform the data into useful information using JavaScript.

Functional Requirements

A. Search for a GitHub Developer

Provide an input where a user can enter a GitHub username and a button to start the search. When the search starts:

- Validate that the username is not empty.
- Show a loading message such as "Loading developer information..."
- Request the profile and repository data.
- Hide or replace the loading message when the requests finish.
- Display the result or a meaningful error message.

A new search should replace the previous result rather than continuously appending duplicate output.

B. Fetch Profile and Repository Data

Create separate asynchronous functions for requesting the two resources. Suggested responsibilities:

```

fetchGitHubUser(username)
fetchUserRepositories(username)

```

The two requests do not depend on each other. Your main application flow should therefore demonstrate `Promise.all()` so both requests can be started together and awaited as a group.

Do not copy the exact implementation from another project. You should decide how your functions validate responses and what they return.

C. Display Developer Profile

After the requests succeed, display at least the following information:

- Avatar
- Name
- GitHub username
- Bio
- Company

- Location
- Followers
- Following
- Public repository count
- Account creation date
- A link to the public GitHub profile

For nullable fields such as name, bio, company, and location, display a friendly fallback such as "Not provided" instead of showing null or undefined.

D. Repository Statistics

Use the repository array to calculate and display the following:

- Number of repositories returned by the API
- Total stars across the returned repositories
- Total forks across the returned repositories
- Most-starred repository
- Five most-starred repositories
- Most frequently used programming language
- A list of the distinct programming languages used

Use array methods rather than hard-coding results. For example, `reduce()` is appropriate for totals, `sort()` can rank repositories, and `Set` can help produce a list of unique languages.

When calculating language statistics, ignore repositories where language is null.

E. Repository Filtering

Provide at least two ways to inspect the fetched repository data. Examples include:

- Show only repositories written primarily in a selected language.
- Show only repositories with at least one star.
- Show repositories that are not forks.
- Sort repositories by stars, name, or most recently updated.

You do not need to make additional API requests for these operations. Perform them on the repository array already stored in memory.

F. Account Age

Use the `created_at` value from the profile response and JavaScript `Date` to calculate the approximate age of the GitHub account. The result can be expressed in years. The goal is to demonstrate that API timestamps are strings that can be converted into `Date` objects and used in calculations.

Asynchronous JavaScript Requirements

This is the most important part of the assignment. Your project must clearly demonstrate the following concepts:

Concept	Required Use
fetch()	Make requests to the GitHub REST API.
Promise	Understand that fetch() and response.json() are asynchronous and return Promises.
async / await	Use async functions for readable asynchronous control flow.
Promise.all()	Fetch the user profile and repositories concurrently.
try...catch	Catch errors from the asynchronous application flow.
throw new Error()	Create meaningful errors for invalid HTTP responses or invalid input.
response.ok	Check whether an HTTP request was successful before treating the response as valid.
response.json()	Parse JSON response bodies into JavaScript values.

HTTP Response Handling

A completed fetch() does not automatically mean the request was successful. A request for a username that does not exist can still produce a Response object. Check the response before using its data.

Your application should distinguish at least these situations:

Situation	Expected Behavior
200 OK	Parse the JSON and continue.
404 Not Found	Display a message such as "GitHub user not found."
403 Forbidden / 429 Too Many Requests	Explain that the GitHub API rate limit may have been reached and stop making unnecessary retries.
Network failure	Catch the error and display a connection/request failure message.
Empty username	Do not call the API; ask the user to enter a username.

GitHub documents a primary limit of 60 REST API requests per hour for unauthenticated public requests, associated with the originating IP address. This is important in a classroom because multiple students may share the same public IP. Do not repeatedly refresh or poll the API during testing.

For this assignment, students should not put personal access tokens into browser-side JavaScript. Authentication and secret management will be more appropriate after moving to backend development.

Required JavaScript Concepts

- `const` and `let`
- Objects and arrays
- Functions and arrow functions
- Template literals
- Destructuring
- Spread operator where appropriate
- Optional chaining (`?.`)
- Nullish coalescing (`??`)
- Array methods: `map()`, `filter()`, `reduce()`, `find()`, `sort()`
- Set or Map where appropriate
- Date
- ES modules: `import` and `export`
- Conditional statements
- Error handling

Suggested Project Structure

```
github-developer-explorer/  
|  
|-- index.html  
|-- js/  
|   |-- app.js  
|   |-- githubApi.js  
|   |-- analytics.js  
|   |-- ui.js  
|  
|-- README.md  
|-- style.css           (optional)
```

Suggested responsibilities:

- `githubApi.js` - functions that communicate with GitHub.
- `analytics.js` - functions that calculate stars, languages, top repositories, and other statistics.
- `ui.js` - functions that display loading states, results, and errors.
- `app.js` - event handling and the main application flow.

You may use a different structure. The important requirement is to avoid placing the entire application in one JavaScript file and to demonstrate ES module import/export.

Minimum User Flow

Your completed application should support this scenario:

1. User opens the application.
2. User enters a GitHub username.
3. Application validates the input.
4. A loading state is displayed.
5. Profile and repository requests begin.
6. The application waits for both requests.
7. Responses are validated and parsed as JSON.

8. Profile information is displayed.
9. Repository statistics are calculated and displayed.
10. The user filters or sorts the repository results.
11. The user can enter another username and repeat the process.

Example Output

GITHUB DEVELOPER EXPLORER

Developer: Example Developer (@example)
Location: Addis Ababa
Followers: 340
Following: 120
Public repositories: 42
Member since: 2020
Account age: approximately 6 years

REPOSITORY ANALYSIS

Repositories returned: 42
Total stars: 318
Total forks: 74
Most used language: JavaScript
Languages: JavaScript, Python, TypeScript, HTML

TOP 5 REPOSITORIES

1. awesome-project	120 stars
2. api-service	64 stars
3. data-tool	51 stars
4. portfolio	43 stars
5. cli-project	20 stars

The exact numbers above are only an example. Your application must display values calculated from the real API response.

Validation Rules

Your application should reject or handle invalid situations instead of crashing. At minimum:

- Do not send a request for an empty username.
- Handle usernames that do not exist.
- Do not assume bio, company, location, description, or language is always present.
- Handle an empty repository array.
- Do not calculate a "most-starred repository" from an empty array without checking first.
- Do not calculate a most-used language when every repository has language: null.
- Display meaningful errors instead of raw stack traces to the user.

Restrictions

- Do not use Axios.
- Do not use Express.js.
- Do not use React, Vue, Angular, or another framework.
- Do not use a database.

- Do not use npm packages or external JavaScript libraries.
- Do not copy a complete GitHub explorer project from the internet.

Use Vanilla JavaScript and the browser Fetch API. CSS may be used for simple presentation, but design is not the main grading criterion.

Bonus Features (Optional)

- Compare two developers: fetch both profiles/repository sets and compare followers, stars, languages, and account age.
- Pagination: retrieve additional repository pages for users with more than 100 public repositories.
- Caching: store fetched results in memory so repeating the same search can reuse previous data during the session.
- Language percentages: calculate what percentage of repositories use each primary language.
- Repository age: use `created_at` to find the oldest and newest repository.
- Activity insight: use `pushed_at` or `updated_at` to identify recently active repositories.
- Loading animation: show a small timer/spinner while the requests are pending.
- Graceful rate-limit information: read `x-ratelimit-remaining` and show the remaining request count for learning purposes.

Submission Instructions

Submit the following:

- A public GitHub repository containing the complete project.
- A README.md with project title, description, features, API endpoints used, folder structure, and instructions for running the project.
- Clean, readable, and modular JavaScript.
- A short 5-10 minute demonstration explaining the asynchronous flow and at least one repository-analysis function.

Commit your work regularly using meaningful Git commit messages.

Evaluation Criteria (100 Marks)

Criteria	Marks
GitHub API Integration	20
Asynchronous JavaScript: <code>async/await</code> , <code>Promise.all()</code> , <code>fetch</code>	25
Repository Data Analysis and Array Methods	20
Error Handling and Validation	15
Code Quality, Modules, and Organization	10
User Output / Usability	5
GitHub Repository and README	5

Important Notes

- The project is about JavaScript logic and asynchronous programming, not visual design.
- Plan the asynchronous flow before writing the UI.
- Do not call the API more often than necessary.
- Never assume API data is complete; public profile fields may be null.
- Do not expose passwords, access tokens, or other secrets in source code or Git commits.
- Read the GitHub API response carefully and use only the fields needed by your application.

Goal

This assignment is designed to strengthen your understanding of asynchronous JavaScript before starting Node.js. The central pattern is: receive input -> make asynchronous requests -> wait for results -> validate HTTP responses -> parse JSON -> transform data -> handle errors -> present a useful result.

The same pattern will appear repeatedly in backend development when your Node.js applications communicate with databases, third-party APIs, authentication services, file systems, and other network services.

Official API References

GitHub REST API - Get a user:

<https://docs.github.com/en/rest/users/users#get-a-user>

GitHub REST API - List repositories for a user:

<https://docs.github.com/en/rest/repos/repos#list-repositories-for-a-user>

GitHub REST API - Rate limits:

<https://docs.github.com/en/rest/using-the-rest-api/rate-limits-for-the-rest-api>